CHANDIGARH UNIVERSITY

Department of Computer Applications

PROJECT REPORT

ON

SOIL MANAGEMENT SYSTEM

Submitted in partial fulfilment of the requirements for

DATABASE MANAGEMENT SYSTEMS (DBMS)

Submitted By

Name: Aditya & Nitin Raj

UID: 25BCA10391 & 25BCA10383

Semester: 3rd – 25BCA 6 “A”

Under the Guidance -

Faculty: Mr. Arvinder Singh Sir



CERTIFICATE

Project Certification

This is to certify that the project report entitled “Soil Management System” submitted by Aditya & Nitin Raj, UID - 25BCA10391 & 25BCA10383 of 25BCA 6 “A” Chandigarh University, is a record of work carried out by the student under my guidance and supervision as part of the Database Management Systems (DBMS) coursework.

Academic Evaluation

The project demonstrates the design and implementation of a basic database-driven web application using HTML, CSS, and MySQL. To the best of my knowledge, the work presented in this report is suitable for academic submission and evaluation.

Approval

Faculty Guide:
Signature:
Date:
Department Seal:

DECLARATION

Student Declaration

I hereby declare that the project entitled "Soil Management System" is my own work, completed as part of the Database Management Systems coursework at Chandigarh University under the guidance of the faculty member named in this report.

Originality

I further declare that this report has not been submitted, either in full or in part, to any other institution or university for the award of any degree, diploma or certificate. Wherever information or concepts from external sources have been used, the sources have been acknowledged in the references.

Signature

Student Name: Aditya & Nitin Raj
UID: 25BCA10391 & 25BCA10383
Signature:
Date:

ACKNOWLEDGEMENT

Acknowledgement

I would like to express my sincere gratitude to my respected faculty member, project guide, and the Department of Computer Applications for providing me with the opportunity to develop this project titled "Soil Management System" as part of my Database Management Systems coursework. This project has been a valuable learning experience, helping me connect classroom concepts with practical application development. This work also helped develop a clearer understanding of how a web application communicates with a relational database. The report records the design decisions and the functions included in the current implementation. The project is intended for academic demonstration and should be evaluated within that limited scope. The application stores information supplied by the user and does not independently verify scientific measurements.

Learning and Technical Support

During its implementation, I learned about database creation, relational table design, primary keys, unique constraints, SQL queries, normalization, data validation, and CRUD operations. I also gained an understanding of how a web-based application communicates with a MySQL database to store, retrieve, update, and delete information. This work also helped develop a clearer understanding of how a web application communicates with a relational database. The report records the design decisions and the functions included in the current implementation. The project is intended for academic demonstration and should be evaluated within that limited scope. The application stores information supplied by the user and does not independently verify scientific measurements.

Acknowledgements

I would also like to acknowledge the developers and documentation communities of HTML, CSS, PHP, MySQL, and Apache for providing useful technologies and learning resources that supported the development of this project. Finally, I am thankful to my friends, classmates, and family members for their encouragement, suggestions, and support throughout the planning, development, and documentation of this project. This work has improved my technical knowledge, problem-solving ability, and confidence in developing database-driven applications. I am grateful to everyone who contributed directly or indirectly to its completion. This work also helped develop a clearer understanding of how a web application communicates with a relational database. The report records the design decisions and the functions included in the current implementation. The project is intended for academic demonstration and should be evaluated within that limited scope. The application stores information supplied by the user and does not independently verify scientific measurements.

ABSTRACT

Project Summary

The Soil Management System is a web-based application developed to store, manage, and maintain soil sample records efficiently. Its main objective is to demonstrate the practical use of Database Management System concepts through a simple interface. Each record includes a unique sample code, farmer name, location, soil type, pH value, optional nitrogen, phosphorus and potassium readings, moisture percentage, recommended crop, sample date, and notes. This work also helped develop a clearer understanding of how a web application communicates with a relational database. The report records the design decisions and the functions included in the current implementation. The project is intended for academic demonstration and should be evaluated within that limited scope. The application stores information supplied by the user and does not independently verify scientific measurements.

Technology and Operations

The system supports Create, Read, Update and Delete (CRUD) operations, allowing users to insert, view, modify and remove records. HTML and CSS are used for the interface, PHP handles server-side processing, and MySQL stores the information. PHP Data Objects (PDO) and prepared statements are used for database communication. Primary keys, unique constraints, validation rules and indexes help maintain data integrity and support efficient searching. This work also helped develop a clearer understanding of how a web application communicates with a relational database. The report records the design decisions and the functions included in the current implementation. The project is intended for academic demonstration and should be evaluated within that limited scope. The application stores information supplied by the user and does not independently verify scientific measurements.

Scope and Outcome

Users can search records by sample code, farmer name, location or soil type. The project is designed for academic learning and demonstrates database design, SQL queries, connectivity and basic application security. Future improvements may include login facilities, user roles, soil analysis reports and historical comparisons. The prototype does not automatically diagnose soil health or provide certified agricultural advice. This work also helped develop a clearer understanding of how a web application communicates with a relational database. The report records the design decisions and the functions included in the current implementation. The project is intended for academic demonstration and should be evaluated within that limited scope. The application stores information supplied by the user and does not independently verify scientific measurements.

TABLE OF CONTENTS

Chapters

1. Introduction
2. Problem Statement
3. Objectives
4. Scope
5. Existing and Proposed System
6. Feasibility Study
7. Requirement Analysis
8. Hardware and Software Requirements
9. System Architecture
10. Data Flow and Process Description
11. Database Design
12. ER Model and Schema
13. Normalization
14. Data Dictionary
15. SQL Implementation
16. Application Modules
17. User Interface Design
18. CRUD Operations
19. Search Functionality
20. Validation and Security
21. Testing Strategy
22. Test Cases and Results
23. Installation and Deployment
24. User Manual
25. Limitations
26. Future Enhancements
27. Maintenance and Scalability
28. Conclusion
29. References
30. Appendices

List of Illustrations

Architecture diagram, entity relationship diagram, database schema, application interface screenshots and sample output tables may be inserted in the relevant chapters after capturing them from the running application.

1. INTRODUCTION

Soil is one of the most important natural resources because it supports plant growth, agriculture, construction planning and many environmental activities. Information about soil can include its type, pH value, nutrient content, moisture level, sample location and the date on which the sample was collected. When this information is recorded properly, it becomes easier to review the condition of different samples and maintain a history of observations. In many small institutions or student projects, these details may be written in notebooks or stored in separate spreadsheet files. Such methods can become difficult to manage when the number of records increases.

The Soil Management System is developed as a simple web-based record-management application. It provides a common place to enter, view, search, update and delete soil sample information. The application uses HTML and CSS to present forms and pages, PHP to process requests, and MySQL to store the records. It is designed as a DBMS academic project, so its main focus is on database concepts and practical implementation rather than scientific soil analysis.

This report explains the problem, objectives, scope, system design, database structure, implementation, testing and possible improvements. It also describes how the application can be installed and used in a local XAMPP environment. The system stores information supplied by the user; it does not independently test soil or certify agricultural recommendations.

1.1 Background

Soil is an important natural resource for agriculture, land planning and environmental management. Soil samples may be tested to understand properties such as acidity or alkalinity, nutrient availability and moisture. Maintaining these observations in a structured form makes it easier to retrieve, review and compare sample information. Soil information can be useful when comparing samples collected from different places or on different dates. Keeping these details in a consistent format makes it easier to review the information and find a particular sample later. The application is a record-keeping tool only; it does not measure soil properties or independently verify the values entered. This distinction is important because the quality of any stored record depends on the accuracy of the original observation or test report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance.

1.2 Project Overview

The Soil Management System is a small database-driven web application. It allows a user to maintain soil sample records through a browser. The application uses PHP as the server-side layer between the user interface and a MySQL relational database. Soil information can be useful when comparing samples collected from different places or on different dates. Keeping these details in a consistent format makes it easier to review the information and find a particular sample later. The application is a record-keeping tool only; it does not measure soil properties or independently verify the values entered. This distinction is important because the quality of any stored record depends on the accuracy of the original observation or test report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance.

1.3 Purpose

The purpose is to provide a simple academic example of how a record-management problem can be addressed using database tables, constraints, SQL statements and a web interface. The project is intentionally limited so that its working can be understood by DBMS students. Soil information can be useful when comparing samples collected from different places or on different dates. Keeping these details in a consistent format makes it easier to review the information and find a particular sample later. The application is a record-keeping tool only; it does not measure soil properties or independently verify the values entered. This distinction is important because the quality of any stored record depends on the accuracy of the original observation or test report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance.

2. PROBLEM STATEMENT

Chapter Overview: The main problem addressed by this project is the difficulty of maintaining soil sample information in an organized and easily searchable manner. When details are stored in paper registers or separate files, a user may need to check many pages or documents to find one sample. Records can also be misplaced, damaged, duplicated or entered in different formats. If a value or location needs to be corrected, the user may have to update more than one copy of the same information. This can create confusion and reduce confidence in the stored data.

A digital record-management system can reduce these difficulties by keeping the information in a structured database. Each sample can be identified by a unique sample code, while fields such as farmer name, location, soil type, pH and sample date provide additional details. Search functions can help users locate records without manually checking every entry. Update and delete operations can also be performed on the selected record.

The project therefore aims to provide a small and understandable solution for storing soil-related records. The expected result is a working prototype that demonstrates how a browser-based form communicates with PHP and a MySQL database. The project does not attempt to replace a soil-testing laboratory, interpret scientific measurements automatically or provide professional crop advice. Its purpose is to improve the organization and retrieval of information entered by the user.

2.1 Problem Identification

When sample information is maintained in notebooks or unstructured files, records may be duplicated, misplaced or difficult to search. Updating an old record can also lead to inconsistent copies when the same information exists in several places. A manual process may work for a few entries, but it becomes less convenient as the number of samples increases. A centralized database reduces the need to search through separate registers and makes the same record available through one interface. Unique sample identification is especially useful when different samples have similar farmer names, soil types or locations. The expected improvement is better organization and retrieval, rather than automatic scientific interpretation. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

2.2 Need for the System

A centralized database provides a consistent place to store sample records. Unique sample codes help identify samples, while search and update operations reduce the effort needed to maintain information. Data validation also helps prevent incomplete or invalid entries. A manual process may work for a few entries, but it becomes less convenient as the number of samples increases. A centralized database reduces the need to search through separate registers and makes the same record available through one interface. Unique sample identification is especially useful when different samples have similar farmer names, soil types or locations. The expected improvement is better organization and retrieval, rather than automatic scientific interpretation. In this project, this topic is considered in relation to the actual requirements of the Soil Management System.

The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

2.3 Expected Outcome

The expected outcome is a working prototype that performs basic record management and demonstrates the interaction between a web form, server-side code and a relational database. A manual process may work for a few entries, but it becomes less convenient as the number of samples increases. A centralized database reduces the need to search through separate registers and makes the same record available through one interface. Unique sample identification is especially useful when different samples have similar farmer names, soil types or locations. The expected improvement is better organization and retrieval, rather than automatic scientific interpretation. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

3. OBJECTIVES OF THE PROJECT

Chapter Overview: The primary objective of the Soil Management System is to design and develop a basic web application that stores and manages soil sample records using database management concepts. The application is intended to demonstrate how a relational database can be connected to a website and used to perform common record-management activities. It also provides practical experience in designing forms, processing input and displaying information retrieved from a database.

The functional objectives include allowing a user to add a new soil sample, display saved samples, edit an existing record and remove a record when required. The system also provides search options using fields such as sample code, farmer name, location and soil type. A unique sample code is used to identify each sample and avoid duplicate codes. Required fields and numeric values are checked so that incomplete or unsuitable input can be rejected before it is stored.

The academic objectives are equally important. Through this project, the student can understand tables, columns, data types, primary keys, unique constraints, indexes and SQL statements. The implementation also introduces PHP database connectivity through PDO and the use of prepared statements. Testing the application helps develop problem-solving skills and shows why data validation is necessary. Overall, the project connects classroom DBMS concepts with a practical application while keeping the system small enough to understand, demonstrate and maintain.

3.1 Main Objective

To design and implement a simple web-based system for storing and managing soil sample records using MySQL. The objective can be checked by confirming that the relevant feature works correctly in the running application. For example, a record-management objective is achieved when valid information can be saved and later retrieved without changing unrelated records. The project also encourages a structured development process in which requirements are identified before coding begins. These objectives provide a basis for selecting test cases and evaluating the completed prototype. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

3.2 Functional Objectives

The system aims to insert new sample records, display saved records, update selected information, delete unwanted records, search records using common fields, validate user input and maintain data integrity. The objective can be checked by confirming that the relevant feature works correctly in the running application. For example, a record-management objective is achieved when valid information can be saved and later retrieved without changing unrelated records. The project also encourages a structured development process in which requirements are identified before coding begins. These objectives provide a basis for selecting test cases and evaluating the completed prototype. In this project, this topic is considered in relation to the actual requirements of the Soil Management System.

The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

3.3 Learning Objectives

The project provides practical experience with relational database design, SQL, primary keys, unique constraints, data types, prepared statements, PHP database connectivity and CRUD operations. The objective can be checked by confirming that the relevant feature works correctly in the running application. For example, a record-management objective is achieved when valid information can be saved and later retrieved without changing unrelated records. The project also encourages a structured development process in which requirements are identified before coding begins. These objectives provide a basis for selecting test cases and evaluating the completed prototype. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

4. SCOPE OF THE PROJECT

Chapter Overview: The scope of the Soil Management System is limited to the digital storage and basic management of soil sample information. The application is designed for use in a local computer environment and provides a simple browser-based interface. A user can enter details such as sample code, farmer name, location, soil type, pH value, optional nitrogen, phosphorus and potassium readings, moisture percentage, recommended crop, sample date and notes. These fields provide a structured way to record information that may otherwise be kept in notebooks or separate files.

The system includes the main database table, data-entry form, record listing, search facility and options to edit or delete saved records. The project uses PHP for server-side processing and MySQL for data storage. Basic validation and database constraints are included to improve the consistency of the stored information. The application can be demonstrated using XAMPP or a similar local PHP and MySQL environment.

Some advanced functions are outside the current scope. The system does not connect to soil-testing equipment, automatically calculate nutrient requirements, authenticate multiple users or provide verified agricultural recommendations. It also does not replace laboratory reports or professional advice. The intended users are students, demonstrators or staff who need to enter and review sample information in a controlled environment. These boundaries keep the project realistic for DBMS coursework and provide clear opportunities for future development.

4.1 Included Features

The prototype includes a single main soil_records table, a browser-based data-entry form, record listing, search, edit and delete functions, and basic validation. The design is suitable for demonstration on a local computer using a PHP/MySQL environment. Defining the scope prevents the project from claiming features that have not been developed or tested. The current implementation is intended for a controlled local environment and a relatively small collection of records. Any feature outside this boundary should be treated as a possible enhancement rather than part of the completed system. This makes the report more accurate and helps the evaluator understand what the application can and cannot do. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

4.2 Exclusions

The prototype does not connect to laboratory instruments, calculate fertilizer doses, authenticate multiple users or certify crop recommendations. These functions are outside the current academic scope. Defining the scope prevents the project from claiming features that have not been developed or tested. The current implementation is intended for a controlled local environment and a relatively small collection of records. Any feature outside this boundary should be treated as a possible enhancement rather than part of the completed system. This makes the report more accurate and helps the evaluator understand what the application can and cannot do. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project.

The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

4.3 Intended Users

The intended users are students, demonstrators or staff who need to enter and review sample soil information in a controlled local environment. Defining the scope prevents the project from claiming features that have not been developed or tested. The current implementation is intended for a controlled local environment and a relatively small collection of records. Any feature outside this boundary should be treated as a possible enhancement rather than part of the completed system. This makes the report more accurate and helps the evaluator understand what the application can and cannot do. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

5. EXISTING AND PROPOSED SYSTEM

Chapter Overview: The existing approach to maintaining soil sample information may involve paper registers, handwritten forms or separate spreadsheet files. These methods are easy to start using, but they can become difficult to manage when the number of samples grows. Searching for one record may take time, and information can be duplicated or entered inconsistently. Updating a record may also require changes in more than one place. Paper documents can be damaged or misplaced, while separate files may not follow the same structure.

The proposed Soil Management System provides a centralized digital method for maintaining these records. It stores each sample as a row in a MySQL database and uses a unique sample code to identify it. A web interface allows the user to add new records, view existing records, update selected details and delete records that are no longer required. Search options make it easier to find information by sample code, farmer name, location or soil type.

The proposed approach improves convenience and consistency, but it still depends on users entering correct information. PHP handles the application logic and communicates with MySQL using PDO prepared statements. The system is intended as an academic prototype and does not perform automatic soil diagnosis. Its main benefit is that it demonstrates how a database-driven application can replace scattered manual records with a structured and searchable collection of information.

5.1 Existing Approach

A manual approach may use paper forms or separate spreadsheets. Such methods are easy to begin with but can make searching, consistency and record maintenance difficult as the number of samples increases. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

5.2 Proposed Approach

The proposed application stores records in a MySQL database and provides a common interface for the four CRUD operations. The database enforces key constraints and the PHP layer validates submitted values. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

5.3 Benefits

The proposed system makes record retrieval more convenient, reduces repeated manual entry, supports consistent sample identification and provides a foundation for future features. It does not eliminate the need for correct data entry by users. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

6. FEASIBILITY STUDY

Chapter Overview: A feasibility study is carried out to understand whether a proposed project can be developed and used with the available resources. The Soil Management System is feasible as a DBMS academic project because it uses commonly available technologies and does not require expensive equipment. HTML and CSS are used for the user interface, PHP is used for server-side processing, and MySQL is used for data storage. These technologies can be installed and tested locally through XAMPP.

Technical feasibility is supported by the availability of a computer capable of running a local web server and database service. The application uses standard web forms and database operations, so it does not require specialized hardware. Economic feasibility is also reasonable because the main development tools are available free of charge. The project can be completed using a normal student development environment without purchasing commercial software.

Operational feasibility concerns whether the intended user can understand and operate the application. The interface is based on familiar actions such as adding, viewing, searching, editing and deleting records. Basic instructions can help a new user complete these tasks. Schedule feasibility is supported by the limited project scope: the system focuses on one main table and essential CRUD operations rather than complex integrations.

The project is therefore suitable for classroom demonstration and learning. However, a production deployment would require additional security, backups, user permissions and operational planning. The feasibility assessment applies to the academic prototype, not to a fully managed agricultural information service.

6.1 Technical Feasibility

The project uses commonly available technologies: Apache, PHP and MySQL. These can be installed together through packages such as XAMPP, making the application practical to run on a student computer. The selected tools are commonly used in introductory web development and database coursework. A student can install the required software on a personal computer and test the application without purchasing a dedicated server. The system is also small enough to be demonstrated during a practical session using sample records. If the application is used beyond a classroom demonstration, additional planning would be needed for security, backups and support. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

6.2 Economic Feasibility

The software stack is available without paid licensing for this educational use. Development costs are therefore limited mainly to the computer and time required for implementation and testing. The selected tools are commonly used in introductory web development and database coursework. A student can install the required software on a personal computer and test the application without purchasing a dedicated server. The system is also small enough to be demonstrated during a practical session using sample records. If the application is used beyond a classroom demonstration, additional planning would be needed for security, backups and support. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

6.3 Operational Feasibility

The interface uses familiar forms, buttons and tables. A user with basic computer knowledge can enter and manage records after a short demonstration. The selected tools are commonly used in introductory web development and database coursework. A student can install the required software on a personal computer and test the application without purchasing a dedicated server. The system is also small enough to be demonstrated during a practical session using sample records. If the application is used beyond a classroom demonstration, additional planning would be needed for security, backups and support. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

7. REQUIREMENT ANALYSIS

Chapter Overview: Requirement analysis identifies what a system must do and the conditions under which it should operate. For the Soil Management System, the main requirement is to provide a simple way to maintain soil sample records in a database. The application should accept sample details through a form, validate the submitted values and save valid information in MySQL. It should also display stored records and provide options to update or delete a selected entry.

The functional requirements include adding a sample, viewing the record list, searching by common fields, editing existing information and deleting a record. The system should identify each sample through a unique sample code. Required fields should not be left empty, and numeric measurements should be checked before saving. The application should show a useful message when an operation succeeds or when input needs correction.

Non-functional requirements describe qualities of the application. The interface should be simple and readable. Database operations should be reliable, and the application should avoid unnecessary duplication of stored information. Prepared statements should be used for queries containing user input, and displayed values should be escaped before being placed in HTML. The application should work in a supported browser while Apache, PHP and MySQL are running locally.

These requirements define the expected behaviour of the prototype and guide its design and testing. They also help distinguish implemented features from possible future improvements. Features such as user accounts, automated soil interpretation and laboratory-device integration are not part of the current version.

7.1 Functional Requirements

The application shall allow the user to create a soil record, view all records, search for records, edit an existing record and delete a selected record. It shall reject missing required fields and duplicate sample codes. The requirements should be checked before installation so that missing services or incompatible versions do not interrupt the demonstration. Apache serves the application pages, PHP processes requests, and MySQL stores the submitted records. A browser is used to interact with the system, while a code editor is useful for maintaining the source files. The exact configuration may vary depending on the operating system and the versions installed on the computer. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

7.2 Non-Functional Requirements

The system should be easy to use, responsive on a typical desktop browser, maintainable by a student developer and reliable for a small academic dataset. Database operations should use prepared statements. The requirements should be checked before installation so that missing services or incompatible versions do not interrupt the demonstration. Apache serves the application pages, PHP processes requests, and MySQL stores the submitted records. A browser is used to interact with the system, while a code editor is useful for maintaining the source files. The exact configuration may vary depending on the operating system and the versions installed on the computer. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

7.3 Data Requirements

Each sample must have a unique identifier and sample code. Soil pH, nutrient values and moisture should use numeric data types. The collection date should use a date-compatible type. The requirements should be checked before installation so that missing services or incompatible versions do not interrupt the demonstration. Apache serves the application pages, PHP processes requests, and MySQL stores the submitted records. A browser is used to interact with the system, while a code editor is useful for maintaining the source files. The exact configuration may vary depending on the operating system and the versions installed on the computer. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

8. HARDWARE AND SOFTWARE REQUIREMENTS

Chapter Overview: The Soil Management System is designed to run on a normal personal computer or laptop. A modern computer with sufficient memory and storage can run the local web server, PHP interpreter and MySQL database together. A keyboard and mouse are useful for entering and managing sample information, while a display is required to view the application pages. No special soil sensor or laboratory device is required because the current project stores information entered manually by the user.

The software environment includes an operating system such as Windows, Linux or macOS, a web browser and a local server package. XAMPP is suitable for a student setup because it provides Apache, PHP and MySQL or MariaDB components in one package. A code editor can be used to create and modify the HTML, CSS and PHP files. phpMyAdmin may be used to create the database, import the SQL file and inspect table contents.

The application requires a browser that supports standard HTML and CSS. PHP must be configured with PDO and the appropriate MySQL database driver. The database service must be running before the application can connect successfully. The configuration file should contain the correct local database name, host, username and password.

These requirements are intentionally modest so that the project can be demonstrated in a college laboratory or on a personal computer. Actual performance depends on the computer, software versions and number of records stored. For a larger or public deployment, the environment would need additional security and maintenance planning.

8.1 Hardware

A basic laptop or desktop computer with a modern processor, at least 4 GB RAM, sufficient free disk space and a keyboard and mouse is adequate for local demonstration. The requirements should be checked before installation so that missing services or incompatible versions do not interrupt the demonstration. Apache serves the application pages, PHP processes requests, and MySQL stores the submitted records. A browser is used to interact with the system, while a code editor is useful for maintaining the source files. The exact configuration may vary depending on the operating system and the versions installed on the computer. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

8.2 Software

The software environment includes a supported operating system, XAMPP or an equivalent Apache/PHP/MySQL stack, a modern web browser, a code editor and phpMyAdmin or a MySQL client for database administration. The requirements should be checked before installation so that missing services or incompatible versions do not interrupt the demonstration. Apache serves the application pages, PHP processes requests, and MySQL stores the submitted records. A browser is used to interact with the system, while a code editor is useful for maintaining the source files. The exact configuration may vary depending on the operating system and the versions installed on the computer. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

8.3 Technologies

HTML structures the forms and tables, CSS styles the interface, PHP handles server-side logic, PDO manages database access, and MySQL stores persistent records. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

9. SYSTEM ARCHITECTURE

Chapter Overview: The system architecture describes how the different parts of the Soil Management System work together. The application follows a simple three-layer arrangement consisting of the presentation layer, application layer and database layer. The presentation layer is the part visible to the user in a web browser. It contains forms, buttons, search fields and tables created with HTML and styled with CSS. The user interacts with this layer to enter new sample information or request an operation.

The application layer is implemented using PHP. When a form is submitted, PHP receives the input, checks the values and decides which database operation is required. It handles actions such as inserting a new sample, retrieving records, updating a selected record or deleting it. PDO is used to establish communication with MySQL, and prepared statements are used when user-supplied values are included in SQL queries.

The database layer is implemented in MySQL. It stores the soil sample details in the soil_records table. The table uses soil_id as its primary key and sample_code as a unique field. Other columns store details such as farmer name, location, soil type, pH, nutrient values, moisture and sample date.

The normal flow begins when a user opens the application in a browser. The browser sends a request to the local web server, PHP processes the request and communicates with MySQL, and the resulting response is displayed in the browser. This separation makes the application easier to understand and maintain.

9.1 Architecture Overview

The application follows a simple three-layer arrangement. The presentation layer is the browser interface, the application layer is PHP, and the data layer is the MySQL database. Soil information can be useful when comparing samples collected from different places or on different dates. Keeping these details in a consistent format makes it easier to review the information and find a particular sample later. The application is a record-keeping tool only; it does not measure soil properties or independently verify the values entered. This distinction is important because the quality of any stored record depends on the accuracy of the original observation or test report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

9.2 Request Flow

A user submits a form in the browser. PHP receives the request, validates the values and executes a prepared SQL statement through PDO. MySQL performs the requested operation and PHP returns a page showing the updated result. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

9.3 Architecture Benefits

Separating the interface, processing logic and database makes the project easier to understand and maintain. It also allows each layer to be improved independently in a future version. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

10. DATA FLOW AND PROCESS DESCRIPTION

10.1 Input

The user enters a sample code and other soil details in the form. Required values are checked before the record is stored. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

10.2 Processing

PHP validates the request, normalizes appropriate text values and prepares the SQL statement. Parameters are bound separately from the SQL command to reduce SQL injection risk. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

10.3 Output

For a successful operation, the application displays a confirmation or refreshed record list. If validation or database processing fails, the user should receive a clear error message. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

11. DATABASE DESIGN

Chapter Overview: Database design defines how information is organized, stored and connected within a database. In the Soil Management System, the database is designed to store soil sample information in a structured relational table. The main table is named soil_records. Each row represents one soil sample, and each column stores a particular attribute of that sample. This arrangement makes it easier to retrieve and update information without keeping separate copies in different files.

The soil_id column is used as the primary key. It is automatically generated for each row and uniquely identifies a database record. The sample_code column is also unique because each soil sample should have its own identification code. Other columns include farmer_name, location, soil_type, ph_value, nitrogen, phosphorus, potassium, moisture, recommended_crop, sample_date and notes. Text, numeric and date data types are selected according to the kind of information stored.

The design allows some measurement fields to remain empty when values are not available. This is useful because not every sample record may contain every nutrient measurement. Required descriptive fields are kept separate from optional values so that the application can validate them appropriately. Timestamp columns can record when a row was created and last updated.

The current project uses one main table to keep the database understandable for a student demonstration. If the application grows, related information such as farmers, locations, laboratories and test results could be moved into separate tables and connected using foreign keys.

11.1 Database Overview

The project uses a relational database named soil_management. Its primary table, soil_records, stores one row for each soil sample. Soil information can be useful when comparing samples collected from different places or on different dates. Keeping these details in a consistent format makes it easier to review the information and find a particular sample later. The application is a record-keeping tool only; it does not measure soil properties or independently verify the values entered. This distinction is important because the quality of any stored record depends on the accuracy of the original observation or test report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

11.2 Primary Key

The soil_id field is the primary key. It uniquely identifies each row and is suitable for internal references even if a sample code or farmer name changes. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in

relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

11.3 Unique Constraint

The sample_code field is unique so that the same sample code cannot be assigned to two different records. This supports reliable identification and reduces accidental duplication. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

11.4 Data Types and Constraints

Text fields store names and notes, decimal fields store measurements, and a date field stores the sample date. Check constraints may restrict pH and moisture to sensible ranges where supported by the database version. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

12. ER MODEL AND SCHEMA DESCRIPTION

12.1 Entity

The current design contains one main entity: SoilRecord. Each instance represents a collected soil sample and its recorded properties. A well-defined structure helps maintain consistent records and makes SQL queries easier to write and understand. The primary key is used for internal identification, while the unique sample code provides a meaningful identifier for the sample. Column names and data types should be selected according to the information being stored and the operations performed on it. If requirements expand, related information can be moved into separate tables and connected through foreign keys. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

12.2 Attributes

The attributes include soil_id, sample_code, farmer_name, location, soil_type, ph_value, nitrogen, phosphorus, potassium, moisture, recommended_crop, sample_date and notes, along with created_at and updated_at timestamps. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

12.3 Relationship Design

The academic prototype uses one table to keep the design straightforward. A future normalized version could separate farmers, locations, laboratories and crop recommendations into related tables using foreign keys. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

13. NORMALIZATION

Chapter Overview: Normalization is a database design process used to reduce unnecessary duplication and improve data consistency. It organizes information into tables and defines suitable relationships between them. In the Soil Management System, the current design uses one main table called soil_records. Each row represents one soil sample, and each column contains one type of information about that sample. For example, farmer name is stored in farmer_name, soil type is stored in soil_type and the pH measurement is stored in ph_value.

The design follows the basic idea of First Normal Form because each field is intended to contain a single value rather than a list of multiple values. The table also has a primary key, soil_id, which uniquely identifies each row. This makes it possible to update or delete a particular sample without depending on descriptive fields that may be repeated.

The table is suitable for a small academic prototype, but a larger application may benefit from further normalization. For example, if the same farmer details are entered for many samples, a separate farmers table could store each farmer once. A farmer_id foreign key could then connect each soil record to the correct farmer. Locations, laboratories and test types could also be separated if the application requires them.

Normalization should be applied according to actual requirements. Too many separate tables can make a small project unnecessarily complicated, while too little structure can cause repeated information and update problems. The current design keeps the project understandable and provides a clear path for future database expansion.

13.1 First Normal Form

The table stores one value in each field and does not use repeating groups of nutrient values. Nitrogen, phosphorus and potassium are represented in separate columns. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

13.2 Second Normal Form

The table uses a single-column primary key, soil_id. Therefore, non-key attributes depend on the complete primary key and there are no partial dependencies. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

13.3 Third Normal Form

The design aims to keep non-key attributes dependent on the key rather than on other non-key attributes. If future requirements introduce repeated farmer or laboratory details, those should be moved to separate related tables to reduce update anomalies. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

14. DATA DICTIONARY

Chapter Overview: A data dictionary provides a clear description of the fields used in a database. It explains the purpose of each column, its data type and any important rule applied to it. In the Soil Management System, the main table is soil_records. The soil_id field is an integer that automatically increases and acts as the primary key. The sample_code field stores a unique text-based identifier for each sample and does not allow duplicate values.

The farmer_name field stores the name associated with the sample, while location stores the place from which the sample was collected. The soil_type field records the entered soil category, such as loamy, sandy or clayey. The ph_value field stores a decimal measurement. Nitrogen, phosphorus and potassium fields store optional nutrient readings, while moisture stores the entered moisture percentage. recommended_crop stores a crop name entered by the user, and sample_date records the date associated with the sample.

The notes field allows additional text to be recorded when necessary. The created_at and updated_at fields store timestamps showing when a record was created and last modified. Data types are selected to suit the information: integer for identifiers, VARCHAR for short text, DECIMAL for measurements, DATE for sample dates and TEXT for longer notes.

The data dictionary is useful during development, testing and maintenance. It helps developers understand the meaning of each field and reduces confusion when writing SQL queries or modifying the database structure. The final dictionary should always match the actual SQL table definition used by the project.

14.1 Main Table: soil_records

soil_id: integer primary key, automatically generated. sample_code: text value, required and unique. farmer_name: name of the farmer or owner. location: sample location. soil_type: soil classification entered by the user. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

14.2 Measurement Fields

ph_value: decimal pH reading. nitrogen, phosphorus and potassium: optional numeric nutrient readings. moisture: numeric moisture percentage. The application should validate values and units before storing them. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

14.3 Other Fields

recommended_crop: crop entered as a recommendation or observation. sample_date: date on which the sample was collected. notes: optional additional comments. created_at and updated_at: timestamps used to track record creation and modification. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

15. SQL IMPLEMENTATION

Chapter Overview: Structured Query Language, commonly called SQL, is used to create and manage relational databases. In the Soil Management System, SQL is used to create the soil_records table and perform the main operations required by the application. The CREATE TABLE statement defines the table name, columns, data types and constraints. The soil_id column is defined as the primary key, while sample_code is given a unique constraint to prevent duplicate sample identifiers.

The INSERT statement is used to add a new soil sample. It stores values supplied through the application form in the appropriate columns. The SELECT statement retrieves records for display or searching. Conditions can be added to a SELECT query to find records by sample code, farmer name, location or soil type. The UPDATE statement changes the details of an existing record, while the DELETE statement removes a selected row.

The PHP application communicates with MySQL using PDO. Prepared statements are used for queries that include values submitted by the user. This approach separates SQL instructions from the data values and helps reduce the risk of SQL injection. The application should also validate information before executing an insert or update operation.

SQL is an important part of this project because it connects the database design with the working application. The report should include the actual SQL table definition and selected query examples used in the implementation. The queries shown in the report should be checked against the final project files so that the documentation remains accurate.

15.1 Database Creation

The database is created using CREATE DATABASE soil_management; and selected using USE soil_management;. The table is created with a CREATE TABLE statement defining the columns, primary key, unique constraint and appropriate data types. A well-defined structure helps maintain consistent records and makes SQL queries easier to write and understand. The primary key is used for internal identification, while the unique sample code provides a meaningful identifier for the sample. Column names and data types should be selected according to the information being stored and the operations performed on it. If requirements expand, related information can be moved into separate tables and connected through foreign keys. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

15.2 Insert Operation

INSERT INTO soil_records (...) VALUES (...); adds a new sample record. The application should use parameterized statements rather than joining user input directly into the SQL string.

15.3 Read, Update and Delete

SELECT retrieves records, UPDATE modifies selected fields, and DELETE removes a chosen record. A WHERE clause should be used for update and delete operations so that only the intended row is affected. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

15.4 Search Query

A search can use conditions such as sample_code LIKE ?, farmer_name LIKE ?, location LIKE ? or soil_type LIKE ?. Search parameters should be passed through PDO binding. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

16. APPLICATION MODULES

Chapter Overview: The Soil Management System is divided into simple modules so that each part has a clear responsibility. The database connection module establishes communication between PHP and MySQL. It uses PDO and the connection details stored in the application configuration. If the database connection is not available, the application should show an appropriate error rather than continuing with an incomplete operation.

The add-record module provides a form for entering new soil sample information. It collects fields such as sample code, farmer name, location, soil type and available measurements. After submission, the PHP code validates the input and inserts the record into the database. The unique sample code constraint helps prevent duplicate sample identifiers.

The record-list module retrieves stored samples and displays them in a table. It allows the user to review the saved information and identify the record that needs attention. The search module filters records using common fields such as sample code, farmer name, location or soil type. The edit module loads the selected record into a form and saves the changed values through an UPDATE query. The delete module removes the selected record after the user confirms the action.

These modules work together to provide the basic record-management functions of the application. Keeping responsibilities separate makes the code easier to understand and troubleshoot. The current version focuses on essential operations rather than advanced services such as login management, laboratory integration or automated soil analysis.

16.1 Database Connection Module

The configuration file stores the database host, database name, username and password required by PDO. Connection errors should be handled without exposing sensitive configuration details to the browser. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

16.2 Record Management Module

This module contains the logic for creating, displaying, editing and deleting soil sample records. Each operation maps to a corresponding SQL statement. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

16.3 Search Module

The search module accepts a keyword and retrieves matching rows from selected text fields. Search results are displayed in a table for convenient review. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

17. USER INTERFACE DESIGN

Chapter Overview: User interface design focuses on how a person interacts with the Soil Management System. The interface should be simple, readable and consistent so that a user can understand the available actions without extensive training. HTML is used to structure page content, forms, labels, buttons and tables. CSS is used to control layout, spacing, colours, borders and text appearance. The design should keep important information visible and avoid unnecessary visual complexity.

The data-entry form is one of the main parts of the interface. It provides labelled fields for the soil sample details and helps the user understand what information is expected. Required fields should be clearly indicated, while optional measurement fields should be identified appropriately. Buttons should use clear action words such as Save, Update, Search and Delete. Validation messages should explain what needs to be corrected rather than displaying confusing technical errors.

The record-list page presents saved samples in a table so that users can review multiple entries. Search controls should be placed near the list to make record retrieval convenient. Edit and delete actions should be associated with the correct row to reduce the chance of changing the wrong sample.

The interface should also remain usable at common screen sizes. Before final submission, screenshots should be captured from the actual running application and inserted into the report with figure numbers and captions. The report should not use sample screenshots that do not match the implemented project.

17.1 Design Principles

The interface is designed to be simple and readable. Labels identify each field, required fields are indicated, and related soil measurements are grouped together. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

17.2 Record Form

The form allows the user to enter sample code, farmer name, location, soil type, pH, nutrient values, moisture, crop recommendation, date and notes. Input types should match the expected values where possible. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

17.3 Record Listing

Saved records are displayed in a table with actions for editing and deleting. Search controls help narrow the displayed records. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

18. CRUD OPERATIONS

Chapter Overview: CRUD represents the four basic operations performed on stored data: Create, Read, Update and Delete. These operations form the central functionality of the Soil Management System. The Create operation allows a user to add a new soil sample. The user completes the form with the sample code and other available details. PHP validates the submitted information and sends an INSERT query to MySQL when the input is acceptable.

The Read operation retrieves saved records from the database. The application uses a SELECT query to display the records in a table. This allows the user to review soil sample information without opening the database directly. The Update operation is used when a stored value needs correction or additional information becomes available. The application loads the selected record, displays its current values and saves the changes using an UPDATE query.

The Delete operation removes a selected record from the database. Since deletion may be difficult to reverse, the interface should ask for confirmation before carrying it out. The application must identify the correct row using its unique soil_id rather than relying only on a name or location.

CRUD operations demonstrate how a web application manages data through a relational database. Each operation should be tested separately using valid and invalid inputs. The result should be checked in both the browser and the database. Correct implementation ensures that an action affects only the intended record and that unrelated records remain unchanged.

18.1 Create

The Create operation inserts a new soil sample record. The application validates required fields and checks for a duplicate sample code before confirming the insert. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

18.2 Read

The Read operation retrieves stored rows and displays them in a table. It may also retrieve a single row when the user opens an edit form. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual

requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

18.3 Update

The Update operation loads the selected record, allows changes and saves them using soil_id as the row identifier. The application should verify that the requested record exists. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

18.4 Delete

The Delete operation removes the selected record after the user confirms the action. A production system should consider soft deletion or audit logging where record history matters. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

19. SEARCH FUNCTIONALITY

Chapter Overview: The search functionality helps users find soil sample records without reviewing the entire database manually. As the number of saved records increases, searching becomes an important feature for convenient data retrieval. In the Soil Management System, the user can search using fields such as sample code, farmer name, location or soil type. These fields are useful because a user may remember one detail about a sample but not its database ID.

When a search request is submitted, PHP receives the keyword and prepares a SELECT query. The query checks the selected fields and returns matching records. Partial matching can be supported so that the user does not always need to enter the complete value. Prepared statements should be used to safely pass the search keyword to the database. The results are then displayed in the record-list table.

Search results should be clear and should show enough information for the user to identify the correct sample. If no matching record is found, the application should display a message explaining that there are no results. The search control should also allow the user to clear the keyword and return to the full record list.

For a small academic database, a straightforward search query is sufficient. If the database becomes much larger, additional improvements such as pagination, more selective indexes and query optimization may be useful. Search behaviour should be tested with complete keywords, partial keywords, empty input and values that do not exist.

19.1 Search Fields

Users can search using sample code, farmer name, location or soil type. This supports common ways of identifying a sample when its internal database ID is unknown. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

19.2 Search Processing

PHP receives the keyword and creates a prepared SELECT query. Wildcard matching can be used for partial matches, and the result set is rendered as rows in the interface. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

19.3 Search Limitations

For a very large database, pagination and more selective indexes may be required. Search results should be limited to avoid loading too many rows at once. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

20. VALIDATION AND SECURITY

Chapter Overview: Validation and security are important because the application accepts information from users and stores it in a database. Validation checks whether submitted values are suitable for the fields in which they are entered. For example, required text fields should not be empty, numeric measurements should contain valid numbers and dates should follow an acceptable date format. Where the project defines permitted ranges, values outside those ranges should be rejected with a clear message.

Server-side validation in PHP is necessary even if the browser form also uses HTML validation. Browser-side checks can improve convenience, but they should not be treated as the only protection because requests can be submitted in other ways. The database also provides important safeguards. A primary key uniquely identifies each row, and a unique constraint prevents repeated sample codes.

PDO prepared statements should be used for SQL queries that contain user-supplied values. This helps prevent SQL injection by keeping query instructions separate from data. Values displayed in HTML should be escaped using an appropriate function to reduce cross-site scripting risks. Forms that change database information should also use CSRF protection where implemented.

The current project is an academic local prototype and does not provide a complete login and role-based access system. It should therefore be demonstrated in a controlled local environment rather than exposed publicly without further security work. Good security also includes protecting configuration details and keeping database backups.

20.1 Input Validation

Required text fields should not be empty. Numeric measurements should be numeric and fall within the accepted range for the project. Dates should be valid dates. Validation should be performed on the server because browser-side checks can be bypassed. Database constraints provide another layer of protection by enforcing rules such as primary-key and unique-value requirements. Prepared statements help protect database queries, while output escaping helps prevent entered text from being interpreted as executable HTML. A local academic prototype should not be exposed to the public internet without a proper review of authentication, permissions and configuration. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

20.2 Prepared Statements

PDO prepared statements separate SQL instructions from user-supplied values. This reduces the risk of SQL injection when implemented consistently. Validation should be performed on the server because browser-side checks can be bypassed. Database constraints provide another layer of protection by enforcing rules such as primary-key and unique-value requirements. Prepared statements help protect database queries, while output escaping helps prevent entered text from being interpreted as executable HTML. A local academic prototype

should not be exposed to the public internet without a proper review of authentication, permissions and configuration. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

20.3 Output Escaping

Values displayed in HTML should be escaped using an appropriate HTML-escaping function to reduce cross-site scripting risks. Validation should be performed on the server because browser-side checks can be bypassed. Database constraints provide another layer of protection by enforcing rules such as primary-key and unique-value requirements. Prepared statements help protect database queries, while output escaping helps prevent entered text from being interpreted as executable HTML. A local academic prototype should not be exposed to the public internet without a proper review of authentication, permissions and configuration. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

20.4 CSRF and Access Control

Forms that change data should use a CSRF token. The current academic prototype does not include user authentication; it should therefore be run locally or in a controlled environment rather than exposed publicly. Validation should be performed on the server because browser-side checks can be bypassed. Database constraints provide another layer of protection by enforcing rules such as primary-key and unique-value requirements. Prepared statements help protect database queries, while output escaping helps prevent entered text from being interpreted as executable HTML. A local academic prototype should not be exposed to the public internet without a proper review of authentication, permissions and configuration. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

21. TESTING STRATEGY

Chapter Overview: Testing is carried out to check whether the Soil Management System works according to its requirements. It helps identify errors before the project is demonstrated or submitted. Testing should cover the main functions as well as the handling of incorrect or incomplete information. Each test should have a clear input, an expected result and an observed result.

Functional testing checks whether the application can add a new soil sample, display saved records, search for matching records, update an existing record and delete a selected record. During these tests, the user should confirm that the correct information appears in the browser and that the database contains the expected changes. It is important to verify that editing one record does not accidentally change another record.

Validation testing checks how the application responds to missing required fields, duplicate sample codes, invalid numeric values and incorrect dates. Search testing should include exact matches, partial matches, empty search input and keywords that do not match any record. Security-related checks should confirm that user values are passed through prepared statements and that displayed text is escaped.

Testing should be performed in the same environment that will be used for the demonstration, such as XAMPP with Apache and MySQL running. Any errors should be recorded and corrected before the final report is prepared. The report should distinguish between tests that were actually performed and tests that are only planned. This keeps the project documentation honest and useful for evaluation.

21.1 Purpose of Testing

Testing checks whether each feature behaves as expected and whether invalid input is handled safely. Both normal and boundary cases should be tested. Soil information can be useful when comparing samples collected from different places or on different dates. Keeping these details in a consistent format makes it easier to review the information and find a particular sample later. The application is a record-keeping tool only; it does not measure soil properties or independently verify the values entered. This distinction is important because the quality of any stored record depends on the accuracy of the original observation or test report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

21.2 Functional Testing

The main functional tests cover record insertion, record listing, searching, editing and deletion. The test process should verify both the browser output and the corresponding database row. A useful test case states the action, the input, the expected result and the actual result observed during execution. The tester should confirm both the message shown on the page and the change made in the database. If the observed result differs from the expected result, the issue should be corrected and the test repeated. Only tests that have actually been performed should be marked as passed in the final submitted report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

21.3 Validation Testing

Validation tests include missing required fields, duplicate sample codes, non-numeric measurements, invalid dates and values outside permitted ranges. Validation should be performed on the server because browser-side checks can be bypassed. Database constraints provide another layer of protection by enforcing rules such as primary-key and unique-value requirements. Prepared statements help protect database queries, while output escaping helps prevent entered text from being interpreted as executable HTML. A local academic prototype should not be exposed to the public internet without a proper review of authentication, permissions and configuration. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

22. TEST CASES AND RESULTS

Chapter Overview: Test cases describe specific actions used to verify that the application behaves as expected. Each test case should include a test number, the feature being tested, the input or action, the expected result and the actual result. For the Soil Management System, the first test should add a valid soil sample using a new sample code. The expected result is that the record is saved and appears in the record list.

A second test should attempt to submit a sample code that already exists. The expected result is that the duplicate is rejected by validation or the database constraint and that no second record with the same code is created. A search test should use a known farmer name or location and verify that matching records are displayed. An update test should change one field, such as the location, and confirm that only the selected record is modified.

A delete test should remove a chosen record after confirmation and verify that the selected row no longer appears. An invalid-input test should leave a required field empty or enter unsuitable numeric information and confirm that the application displays an appropriate message. These tests cover normal operations as well as common errors.

The actual result column must be completed after running the application. If a test fails, the reason and correction should be recorded. Screenshots may be added as evidence for important test cases. This makes the testing chapter more reliable and shows that the system was checked rather than simply described.

22.1 Sample Test Cases

Test 1: submit a valid sample record; expected result: record is inserted. Test 2: submit an existing sample code; expected result: duplicate is rejected. Test 3: search by farmer name; expected result: matching records are displayed. A useful test case states the action, the input, the expected result and the actual result observed during execution. The tester should confirm both the message shown on the page and the change made in the database. If the observed result differs from the expected result, the issue should be corrected and the test repeated. Only tests that have actually been performed should be marked as passed in the final submitted report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

22.2 Update and Delete Tests

Test 4: edit the location of an existing record; expected result: only the selected row is updated. Test 5: delete a selected record; expected result: the selected row is removed and no unrelated row is affected. A useful test case states the action, the input, the expected result and the actual result observed during execution. The tester should confirm both the message shown on the page and the change made in the database. If the observed result differs from the expected result, the issue should be corrected and the test repeated. Only tests that have actually been performed should be marked as passed in the final submitted report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

22.3 Error Tests

Test 6: submit an empty required field; expected result: validation message. Test 7: enter invalid numeric data; expected result: the input is rejected. Actual outcomes should be filled in after running the project on the target computer. A useful test case states the action, the input, the expected result and the actual result observed during execution. The tester should confirm both the message shown on the page and the change made in the database. If the observed result differs from the expected result, the issue should be corrected and the test repeated. Only tests that have actually been performed should be marked as passed in the final submitted report. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

23. INSTALLATION AND DEPLOYMENT

Chapter Overview: Installation and deployment describe the steps required to prepare the environment and run the Soil Management System. For a local academic demonstration, XAMPP can be used because it provides the Apache web server, PHP and MySQL or MariaDB database service. After installing XAMPP, the user should open its control panel and start Apache and MySQL. Both services must be running before the application can be accessed through the browser.

The project folder should be copied into the web server document directory, which is commonly named htdocs in XAMPP. The database should then be created using phpMyAdmin. The supplied SQL file can be imported to create the soil_records table and its constraints. The user should check that the table appears correctly and that the database name matches the value used in the application configuration.

The database connection file should contain the correct host, database name, username and password for the local environment. In a default local XAMPP setup, the host is commonly localhost, but the correct credentials depend on the installation. Once the configuration is checked, the user can open a browser and visit the local project address, such as http://localhost/soil_management/.

After the home page loads, the main functions should be tested. The user should add a sample record, view it, search for it, update a field and test deletion. Deployment beyond a local computer requires additional security, access control, backups and server configuration.

23.1 Environment Setup

Install XAMPP or an equivalent package containing Apache, PHP and MySQL. Start the Apache and MySQL services from the control panel. Before starting, confirm that the required local services are running and that the application configuration matches the database settings. The user should follow each step in order and check the displayed result before moving to the next operation. If an error occurs, check the server status, database name, connection credentials and spelling of the local project address. For important data, keep a database backup before performing bulk changes or deletion. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

23.2 Database Setup

Open phpMyAdmin, create or import the supplied soil_management.sql file, and confirm that the soil_records table has been created. A well-defined structure helps maintain consistent records and makes SQL queries easier to write and understand. The primary key is used for internal identification, while the unique sample code provides a meaningful identifier for the sample. Column names and data types should be selected according to the information being stored and the operations performed on it. If requirements expand, related information can be moved into separate tables and connected through foreign keys. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable,

practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

23.3 Application Setup

Copy the project folder into the web server document root, commonly htdocs in XAMPP. Update config.php with the correct database connection values. Before starting, confirm that the required local services are running and that the application configuration matches the database settings. The user should follow each step in order and check the displayed result before moving to the next operation. If an error occurs, check the server status, database name, connection credentials and spelling of the local project address. For important data, keep a database backup before performing bulk changes or deletion. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

23.4 Running the Application

Open a browser and visit the local project address, for example http://localhost/soil_management/. Confirm that the home page loads and test the CRUD features. Before starting, confirm that the required local services are running and that the application configuration matches the database settings. The user should follow each step in order and check the displayed result before moving to the next operation. If an error occurs, check the server status, database name, connection credentials and spelling of the local project address. For important data, keep a database backup before performing bulk changes or deletion. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

24. USER MANUAL

Chapter Overview: The user manual explains how to operate the Soil Management System. Before using the application, the user should make sure that the local web server and database services are running. The application can then be opened through its local address in a web browser. The home page or record-list page provides access to the main record-management functions.

To add a record, the user should open the add-sample form and complete the required fields. A new sample code should be entered for each sample. The farmer name, location and soil type should be provided, while measurement fields can be completed when the information is available. After checking the entries, the user can submit the form. A success or validation message should indicate the result.

To view records, the user can open the record list. To search, the user should enter a keyword related to the sample code, farmer, location or soil type and submit the search. The matching records should appear in the list. To edit a record, the user should select the edit option beside the correct row, change the required values and save the form. The updated information should then be checked in the list.

To delete a record, the user should select the delete option for the intended row and confirm the action if prompted. Deletion should be performed carefully because it removes stored information. Users should avoid entering real sensitive information into an unsecured demonstration environment and should maintain backups when records are important.

24.1 Adding a Record

Open the application, complete the soil sample form and submit it. Use a new sample code for each sample. Check that all required fields are completed. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

24.2 Viewing and Searching

Use the record list to view saved samples. Enter a sample code, farmer name, location or soil type in the search field to find matching records. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in

relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

24.3 Editing a Record

Choose the edit action beside the required row, change the appropriate values and save the form. Verify the updated values in the record list. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

24.4 Deleting a Record

Choose the delete action for the selected row and confirm the operation. Deletion should be used carefully because it removes the stored record. Each operation should use the correct record identifier so that the intended row is affected and other rows remain unchanged. User-supplied values should be passed through prepared statements instead of being joined directly into SQL strings. After an operation, the application should provide a clear message and display the updated information where appropriate. Testing should include valid values, missing values, duplicate identifiers and search terms that do not return a match. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

25. LIMITATIONS

25.1 Functional Limitations

The system is a small prototype and supports only basic soil record management. It does not include laboratory integration, automatic reports, advanced analytics or multi-user workflows. Any improvement should be introduced only after the existing functions have been tested and documented. A future version should preserve the reliability of the basic record operations while adding new features in a controlled way. Changes to the database structure should be accompanied by updates to the PHP code, interface and test cases. The system should continue to make a clear distinction between user-entered information and any verified external result. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

25.2 Security Limitations

The project is intended for local academic demonstration. It does not provide a full authentication and authorization system, production-grade logging, encrypted backups or deployment hardening. Validation should be performed on the server because browser-side checks can be bypassed. Database constraints provide another layer of protection by enforcing rules such as primary-key and unique-value requirements. Prepared statements help protect database queries, while output escaping helps prevent entered text from being interpreted as executable HTML. A local academic prototype should not be exposed to the public internet without a proper review of authentication, permissions and configuration. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

25.3 Domain Limitations

The application stores user-entered values but does not validate the scientific correctness of laboratory measurements or provide certified agronomic advice. Any improvement should be introduced only after the existing functions have been tested and documented. A future version should preserve the reliability of the basic record operations while adding new features in a controlled way. Changes to the database structure should be accompanied by updates to the PHP code, interface and test cases. The system should continue to make a clear distinction between user-entered information and any verified external result. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

26. FUTURE ENHANCEMENTS

Chapter Overview: The Soil Management System can be improved in several ways if it is developed further. One possible enhancement is to introduce user authentication. Different user roles could be created for administrators, laboratory staff and general users. Each role could be given permissions according to its responsibilities. This would help protect records and make the system more suitable for use by a group.

The database could also be expanded into multiple related tables. For example, farmer information, land locations, laboratories and individual soil tests could be stored separately. Primary and foreign keys would connect the tables and reduce repeated information. This would make the database more flexible when the number of records increases.

Another improvement could be the addition of printable reports. A user might generate a report for one sample or compare multiple samples from the same location. Charts could display recorded pH, moisture or nutrient values over time. Such visualizations would help users review stored information, although scientific interpretation should still be provided by qualified professionals.

Other possible features include importing and exporting records, mobile-friendly forms, pagination, backup scheduling and notifications. A future version could also support controlled import of laboratory results, provided the data format and source are verified. These enhancements should be planned according to actual user needs. The project should first maintain reliable basic operations before adding more complex functions. Each new feature would require database changes, interface updates and additional testing.

26.1 User Accounts

Authentication and role-based permissions could distinguish administrators, laboratory staff and general users. Each user could be restricted to permitted operations. Defining the scope prevents the project from claiming features that have not been developed or tested. The current implementation is intended for a controlled local environment and a relatively small collection of records. Any feature outside this boundary should be treated as a possible enhancement rather than part of the completed system. This makes the report more accurate and helps the evaluator understand what the application can and cannot do. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

26.2 Extended Database

Separate tables could be introduced for farmers, land parcels, laboratories, sample tests and crop recommendations. Foreign keys would connect these entities. A well-defined structure helps maintain consistent records and makes SQL queries easier to write and understand. The primary key is used for internal identification, while the unique sample code provides a meaningful identifier for the sample. Column names and data types should be selected according to the information being stored and the operations performed on it. If requirements

expand, related information can be moved into separate tables and connected through foreign keys. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

26.3 Reports and Analytics

The system could generate printable sample reports, charts of pH and nutrient readings, and historical comparisons for the same land location. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

26.4 Integration

Future versions could support import/export, mobile-friendly forms, laboratory data import and notification features. This section contributes to the overall description of the application and explains how the related part supports the project objectives. The implementation should be consistent with the actual source code and database structure used for the demonstration. The information should be reviewed during testing so that the report accurately represents the completed work. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters. During development, the feature should be checked with both normal input and unsuitable input so that unexpected behaviour can be identified.

27. MAINTENANCE AND SCALABILITY

Chapter Overview: Maintenance keeps the Soil Management System reliable after it has been installed. Basic maintenance includes checking that Apache and MySQL start correctly, reviewing application errors and confirming that the database remains accessible. The database should be backed up regularly, especially if the stored records are important. A backup is useful only if it can be restored, so the restore process should also be tested.

Data maintenance includes checking for incomplete or incorrect records and correcting them through the application. Users should follow a consistent method for assigning sample codes and entering dates, soil types and measurement values. Old records should be retained or removed according to the requirements of the institution or project. Configuration details should be protected and should not be shared unnecessarily.

Scalability refers to the ability of the system to handle growth in records, users or features. If the number of soil records increases, indexes on frequently searched fields can improve retrieval. Pagination can prevent the application from loading an excessively large number of rows on one page. Queries should be reviewed for efficiency, and the database should be monitored as usage grows.

If the project becomes a larger application, its design could be separated into clearer modules and related tables could be introduced. A managed database, user authentication and stronger access controls may also become necessary. Improvements should be based on measured requirements rather than added without a clear purpose. Regular maintenance and careful planning can help the application remain understandable and dependable.

27.1 Data Maintenance

Regular database backups should be taken and restore procedures should be tested. Old records should be reviewed according to the institution's retention requirements. Any improvement should be introduced only after the existing functions have been tested and documented. A future version should preserve the reliability of the basic record operations while adding new features in a controlled way. Changes to the database structure should be accompanied by updates to the PHP code, interface and test cases. The system should continue to make a clear distinction between user-entered information and any verified external result. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

27.2 Performance

Indexes on frequently searched fields can improve retrieval. Pagination, query optimization and database monitoring become important as the number of records increases. Any improvement should be introduced only after the existing functions have been tested and documented. A future version should preserve the reliability of the basic record operations while adding new features in a controlled way. Changes to the database structure should be accompanied by updates to the PHP code, interface and test cases. The system should continue to make a clear distinction between user-entered information and any verified external result. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve. The implementation should remain consistent with the database table, application pages and functions described in the other chapters.

27.3 Scaling the Design

If usage grows, the application can be separated into clearer service layers, use a managed database, add authentication and introduce related tables. Scaling should be based on measured needs rather than adding unnecessary complexity. Separating these responsibilities makes it easier to locate an error and understand which part of the application needs attention. The user interface should not directly manage database storage; requests are handled by PHP before the database is accessed. The database response is then returned to the application and presented in a readable form. This arrangement is suitable for a small project because it demonstrates the basic interaction between a browser, server-side code and a relational database. In this project, this topic is considered in relation to the actual requirements of the Soil Management System. The purpose is to keep the application understandable, practical and suitable for a student-level database project. The information entered by the user is processed by the PHP application and stored in the MySQL database when it passes the required checks. A clear approach is important because it helps reduce confusion during development, demonstration and later maintenance. The design also supports the main goal of keeping soil sample information organized and easy to retrieve.

28. CONCLUSION

Chapter Overview: The Soil Management System demonstrates how a database-driven web application can be used to organize and manage soil sample information. The project provides a structured method for storing details such as sample code, farmer name, location, soil type, pH value, nutrient readings, moisture, sample date and notes. Instead of maintaining information only in paper registers or separate files, users can work with records through a browser-based interface connected to a MySQL database.

The application demonstrates the four basic CRUD operations: Create, Read, Update and Delete. It also includes search functionality to help users find records using common fields. HTML and CSS are used to build the interface, PHP handles application processing, and MySQL stores the information. PDO prepared statements, database constraints and input validation help improve the reliability of database operations.

The project provides practical learning in relational database design, SQL queries, primary keys, unique constraints, data types, PHP connectivity and testing. It also shows the importance of clear requirements, user-friendly forms and careful validation. The current version is intentionally limited and is suitable as an academic prototype.

In conclusion, the project meets its educational purpose by connecting DBMS theory with a working record-management application. It should not be considered a soil diagnosis tool or a source of certified agricultural advice. Future versions could include user accounts, extended database relationships, reports and verified laboratory integrations. These improvements would build on the basic structure developed in this project.

Project Outcome

The Soil Management System demonstrates how a small web application can use a relational database to manage soil sample records. It implements the fundamental CRUD operations and provides search functionality through a browser-based interface.

Learning Outcome

The project connects theoretical DBMS concepts with practical implementation. It provides experience with table design, primary keys, unique constraints, SQL queries, PHP database connectivity, validation and prepared statements.

Final Remarks

Although the current version is intentionally limited, it provides a useful foundation for more advanced agricultural record-management applications. The prototype should be treated as an educational information-management tool rather than a soil diagnosis or advisory service.

29. REFERENCES

Technical Documentation

1. PHP Documentation, https://www.php.net/docs.php
2. MySQL Reference Manual, https://dev.mysql.com/doc/
3. Apache HTTP Server Documentation, https://httpd.apache.org/docs/
4. MDN Web Docs – HTML, CSS and Web Forms, https://developer.mozilla.org/

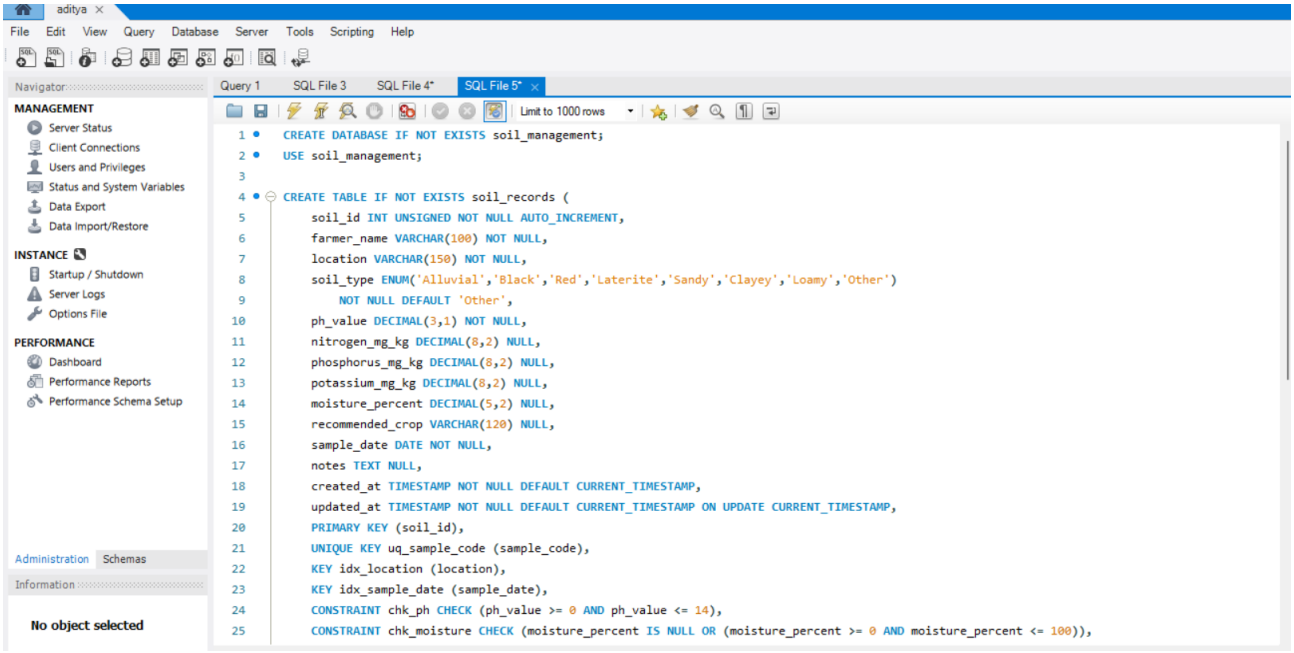
Development Tools

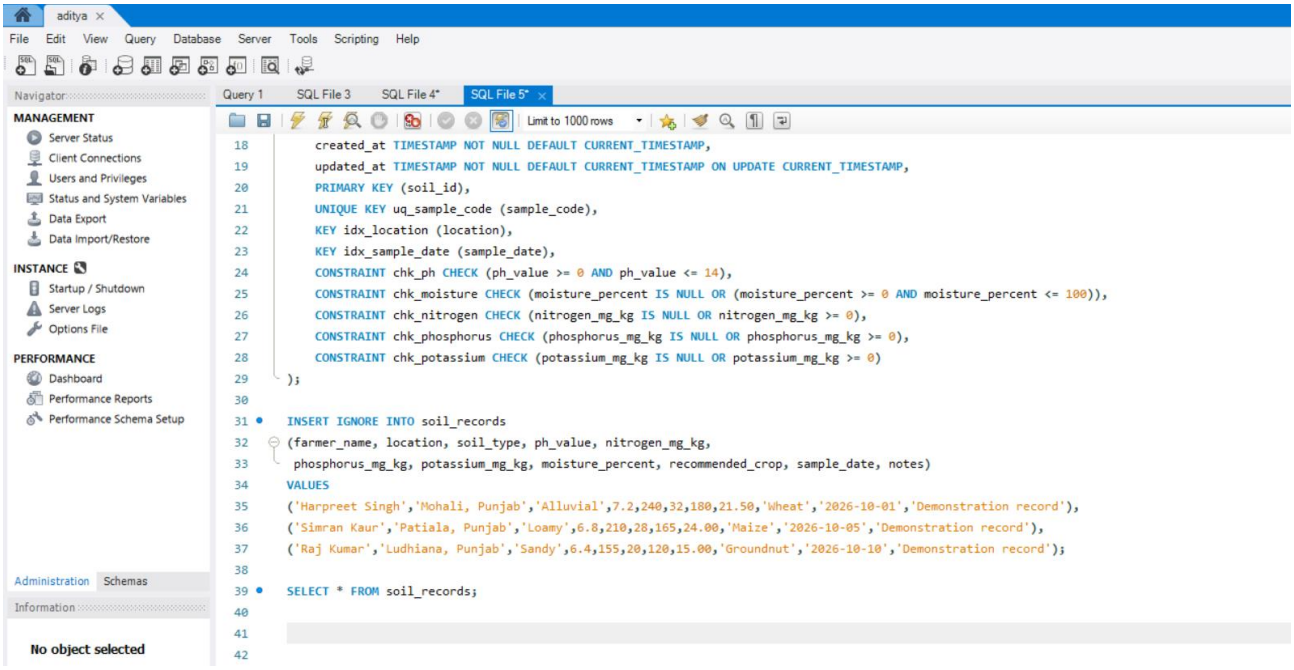
5. XAMPP Documentation and FAQs, https://www.apachefriends.org/
6. PHP Data Objects (PDO) documentation, https://www.php.net/manual/en/book.pdo.php

Academic Note

The references above are general technical documentation sources. Add any textbooks, lecture notes or other materials actually consulted during the project, following the citation style prescribed by the department.

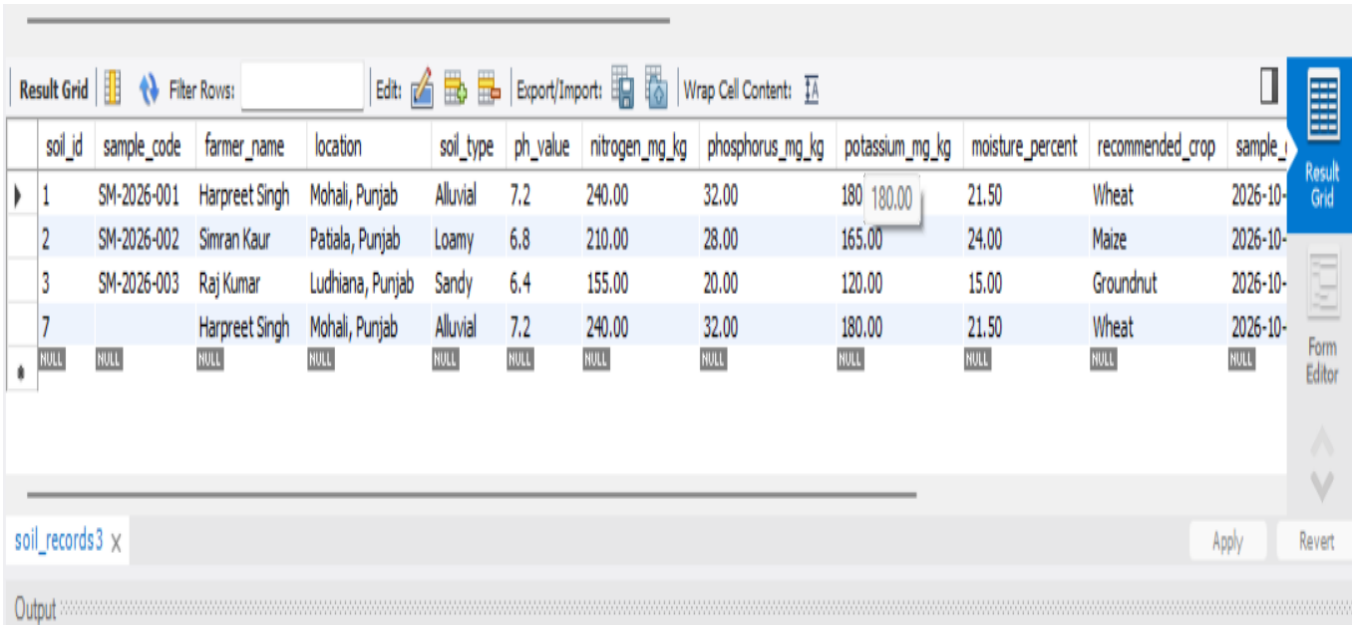
APPENDIX A: SAMPLE SQL QUERIES





APPENDIX B: SCREENSHOTS AND OUTPUT

Screenshots



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

soil_id	sample_code	farmer_name	location	soil_type	ph_value	nitrogen_mg_kg	phosphorus_mg_kg	potassium_mg_kg	moisture_percent	recommended_crop	sample_
1	SM-2026-001	Harpreet Singh	Mohali, Punjab	Alluvial	7.2	240.00	32.00	180 180.00	21.50	Wheat	2026-10-
2	SM-2026-002	Simran Kaur	Patiala, Punjab	Loamy	6.8	210.00	28.00	165.00	24.00	Maize	2026-10-
3	SM-2026-003	Raj Kumar	Ludhiana, Punjab	Sandy	6.4	155.00	20.00	120.00	15.00	Groundnut	2026-10-
7		Harpreet Singh	Mohali, Punjab	Alluvial	7.2	240.00	32.00	180.00	21.50	Wheat	2026-10-
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor

soil_records3 x | Apply | Revert

Output

APPENDIX C: GLOSSARY

Database Terms

DBMS: Database Management System. SQL: Structured Query Language. CRUD: Create, Read, Update and Delete. Primary Key: a field or combination of fields that uniquely identifies a row. Unique Constraint: a rule that prevents duplicate values in a column or column group.

Application Terms

PHP: server-side scripting language. PDO: PHP Data Objects, a database access interface. HTML: HyperText Markup Language. CSS: Cascading Style Sheets. MySQL: relational database management system. Index: a database structure that can improve data retrieval performance.

END OF REPORT